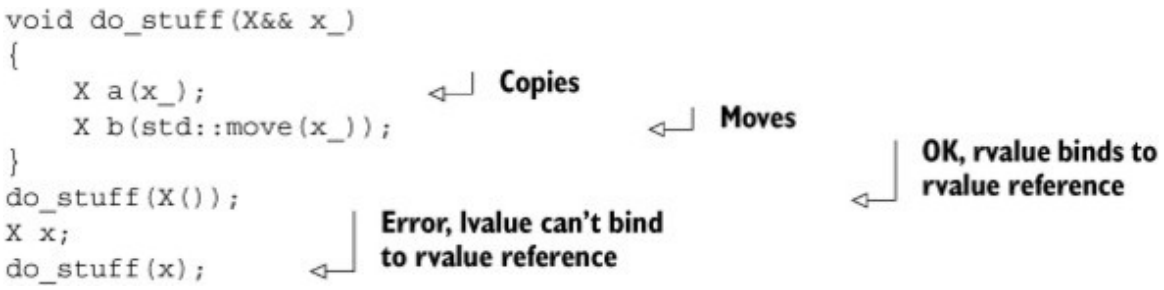
`static_cast<X&&>` or by calling `std::move()`:

```
X x1;
X x2=std::move(x1);
X x3=static_cast<X&&>(x2);
```

This can be beneficial when you wish to move the parameter value into a local or member variable without copying, because although an rvalue reference parameter can bind to rvalues, within the function itself it is treated as an lvalue:



Move semantics are used extensively in the Thread Library, both where copies make no semantic sense but resources can be transferred, and as an optimization to avoid expensive copies where the source is going to be destroyed anyway. You saw an example of this in section 2.2 where we used `std::move()` to transfer a `std::unique_ptr<>` instance into a newly constructed thread, and then again in section 2.3 where we looked at transferring ownership of threads between `std::thread` instances.

None of `std::thread`, `std::unique_lock<>`, `std::future<>`, `std::promise<>`, or `std::packaged_task<>` can be copied, but they all have move constructors to allow the associated resource to be transferred between instances and support their use as function return values. `std::string` and `std::vector<>` both can be copied as always, but they also have move constructors and move-assignment operators to avoid copying large quantities of data from an rvalue.

The C++ Standard Library never does anything with an object that has been explicitly moved into another object, except destroy it or assign *to* it (either with a copy or, more likely, a move). However, it's good practice to ensure that the invariant of the class encompasses the moved-from state. A